

MADai

User Manual

A detailed guide to what the autonomous AI operations platform does, how the pieces fit together, and how operators use it.

Document	Scope	Version
User Manual	MADai application, API, workers, operations, and MADProspects ecosystem	Prepared 2026-05-26

Important security note

This document intentionally does not print any production password, API key, SMTP secret, database credential, or FTP credential. Operational users should obtain credentials through approved MADProspects channels.

1. Executive Overview

MADai is an autonomous AI operations platform for the MADProspects ecosystem. Its core purpose is to accept AI work requests, store and manage those requests as trackable tasks, assign them to registered worker processes, collect progress and artifacts, and provide operators with real-time visibility into the health of the system.

The application is built around a .NET 8 API, SQL Server persistence, Hangfire-ready background processing, SignalR real-time updates, and an Angular frontend. It also includes a separate Claude operator queue used by SystemAdmin users and scheduled Claude/Codex workers to improve the application itself.

Current UI status

The current deployed web frontend is a restored lightweight shell with a home page, login page, and operator console at /app/claude. The backend API and domain model contain a broader operations platform: task queues, workers, dashboards, admin, audit, files, notifications, webhooks, settings, templates, and Claude tasks.

1.1 Main Outcomes

- Turn ad hoc AI requests into managed, auditable work items.
- Run work through registered desktop or server workers using secure worker API keys.
- Track task state from draft, queued, claimed, running, completed, failed, retried, cancelled, and dead-lettered.
- Expose real-time task, worker, notification, and dashboard events through SignalR hubs.
- Support SystemAdmin operations, role-based permissions, feature flags, settings, and company administration.
- Provide a Claude/Codex operator system for application self-improvement work.

1.2 Application Surfaces

Surface	What it is for	Primary users
Frontend	Browser entry point for login and operator workflows. The current shell verifies authentication and SystemAdmin access to /app/claude.	Operators, SystemAdmins
REST API	Main application contract for authentication, tasks, workers, dashboards, audit, files, notifications, admin, settings, webhooks, and Claude tasks.	Frontend, automation, integrations
SignalR hubs	Pushes live task, worker, notification, and dashboard events to connected clients.	Frontend dashboards and real-time clients
Worker API	Allows registered workers to heartbeat, claim tasks, report progress, log output, complete work, or fail work.	MADai workers and trusted automation
Claude operator queue	Separate SystemAdmin queue for repository and product-improvement tasks executed by Claude/Codex schedulers.	SystemAdmins, scheduled AI workers
Deployment scripts	Publish API and frontend to 1-grid/Plesk using FTP and environment injection.	Developers, operators

2. Users, Roles, and Access

MADai uses ASP.NET Identity for users and roles. Human users authenticate with email and password, receive JWT access tokens and refresh tokens, and access endpoints according to their roles and permissions. Workers authenticate separately using X-API-Key so automated execution does not depend on a human JWT.

2.1 Seeded Roles

Role	Typical purpose
SystemAdmin	Full platform administration, Claude task management, system settings, feature flags, plans, companies, and operational control.
CompanyAdmin	Company-level administration for tenant users, company settings, and business operations.
CompanyManager	Operational management for company users and task workflows.
Worker	Worker process identity and queue execution role.
User	Standard authenticated user role.
Client	External or customer-facing user role for limited interaction.
ReadOnly	View-focused role for auditing, review, or support.

2.2 Permission Areas

Area	Permissions represented in the platform
Tasks	View, create, update, delete, cancel, retry, and assign tasks.
Workers	View, manage, and drain worker nodes.
Companies	View and manage tenant companies.
Users	View and manage platform users.
Audit	View and manage audit findings.
System	System management, feature flags, and settings.

2.3 Login Flow

- 1 The user opens the frontend and signs in with an email address and password.
- 2 The API validates the user, checks account status, applies lockout rules, and verifies the password.
- 3 If MFA is enrolled, the API requires a valid authenticator code.
- 4 The API records login history, updates last-login metadata, and issues an access token plus refresh token.
- 5 The frontend stores the session locally and uses the access token on later API calls.
- 6 On logout, the refresh token is revoked so it cannot be reused.

2.4 MFA and Password Management

MADai includes API operations for starting MFA enrollment, verifying MFA setup, disabling MFA, requesting a password reset, resetting a password, changing a password, and updating profile fields. Password reset emails are generated by the application service and delivered through the configured email sender.

2.5 Superadmin Account

The deployment is configured so DEV and PROD have a SystemAdmin account using the approved MADProspects administrator email. The seed process creates the account if it does not exist, confirms the email, ensures it is active, attaches it to the demo company, grants the SystemAdmin role, and resets the password when it differs from the canonical seed. Password values are intentionally excluded from this manual.

3. Task Management

The task system is the operational center of MADai. A task stores the title, description, category, priority, status, queue name, scheduling information, prompt payload, input payload, output summary, result payload, error details, progress, retry counts, dependencies, comments, logs, artifacts, executions, assignments, and tags.

3.1 Task Categories

Category group	Supported categories
Software and delivery	SoftwareGeneration, AngularGeneration, DotNetGeneration, SqlGeneration, Debugging, Testing, Deployment, Documentation, IntegrationBuilding
Creative and content	BlogWriting, BookGeneration, ImageGeneration, VideoGeneration, SocialContent, UxUi
Operations and intelligence	Research, Automation, Scraping, Analysis, Transcription, Reporting, AiOrchestration
Platform recovery	SelfHealing

3.2 Task Status Lifecycle

Status	Meaning
Draft	A task exists but is not yet queued for execution.
Queued	Ready for a worker to claim.
Scheduled	Queued for future execution based on ScheduledAt.
Claimed	Reserved by a worker with a claim token.
Running	A worker has started and is reporting progress.
Paused	Temporarily stopped.
AwaitingValidation	Completed by a worker but not yet accepted as final.
Completed	Finished successfully.
Failed	Failed without being dead-lettered.
Cancelled	Cancelled by a user or operator.
TimedOut	Exceeded allowed execution time.
DeadLetter	Exceeded retry policy or considered terminal.
Recovered	Recovered by audit or operator intervention.

3.3 Priority and Queueing

Tasks are prioritized using Lowest, Low, Normal, High, and Critical. Workers claim eligible queued or due scheduled tasks ordered by priority and creation date. Queue names allow different kinds of work to be routed to different workers. A worker can default to its configured queue or claim from a requested set of queues.

3.4 Dependencies

Tasks may depend on other tasks. Before a worker can claim a task, MADai checks whether hard dependencies have completed. This supports multi-step pipelines where research, drafting, validation, deployment, and reporting can be represented as separate but linked work items.

3.5 Retries and Dead Letters

When a worker reports a transient failure, the platform can requeue the task with exponential backoff. The current retry delay starts from seconds and grows up to a capped delay. Once retries are exhausted, the task can be marked as dead-lettered, preserving the failure reason and making it visible for operator action.

3.6 Comments, Logs, Artifacts, and Templates

- Task comments allow collaboration or operator notes on a task.
- Task logs capture worker-provided log entries and execution detail.
- Task artifacts store files or outputs produced by work.
- Task templates allow common task patterns to be reused.
- Task recommendations can be generated, applied, or dismissed as operational suggestions.

4. Worker Operations

Workers are execution agents. They are registered separately from humans, receive an API key, heartbeat to the API, claim work, report progress, send logs, complete tasks, or fail tasks. Worker API keys are hashed in storage, and workers use the X-API-Key header for subsequent calls.

4.1 Worker Registration and Identity

- 1 A worker registers with metadata such as name, machine name, operating system, queue, capability labels, workspace root, and maximum concurrency.
- 2 The API returns a worker ID and one-time API key.
- 3 The worker stores the key and uses it for heartbeat and task execution calls.
- 4 MADai stores a hash of the key and later authenticates the worker through the Worker authentication scheme.

4.2 Heartbeats and Metrics

Heartbeats update LastHeartbeatAt, worker status, active task count, and optional CPU, memory, disk, and notes. This feeds worker health views and real-time worker update events.

4.3 Claim, Progress, Complete, Fail

Step	What happens
Claim	Worker requests up to MaxTasks. MADai filters by queue, company, schedule, retry timing, dead-letter status, dependency completion, priority, and worker capacity.
Progress	Worker reports progress from 0 to 100 and may attach status text or checkpoint JSON.
Log	Worker sends execution log entries with timestamp, level, message, and source.
Complete	MADai marks the task Completed or AwaitingValidation, stores output summary, result payload, validation report, and frees worker concurrency.
Fail	MADai records failure details, decides retry vs terminal failure, updates retry timing, and frees worker concurrency.

4.4 Worker Statuses

Workers can be Offline, Starting, Idle, Busy, Draining, Errored, or Maintenance. Admin endpoints allow operators to drain or deactivate workers.

5. Claude Operator Queue

MADai has a separate Claude task system for SystemAdmin application-improvement work. This is intentionally different from the end-user TaskItem queue. ClaudeTask records represent developer or operator work items for the repository itself, such as fixes, refactors, setup work, deployment tasks, or scanner findings.

5.1 Claude Task Capabilities

- List, view, create, update, delete, and bulk-import Claude tasks.
- Fetch the next task for a worker or scheduled automation.

- Attach files to Claude tasks and delete attachments.
- Use prompt templates for repeatable operator instructions.
- Control state transitions through a state-machine gate.
- Receive real-time task update events through the SignalR infrastructure.

5.2 Statuses and Priorities

Claude tasks support statuses such as Pending, InProgress, ToBeDeployed, Completed, Failed, Cancelled, and Deferred. Priorities allow operators to order work and control attention.

5.3 Scheduled Worker and Scanner

The README describes a scheduled worker that polls for Claude work and a scanner that imports findings. The worker uses adaptive backoff so a busy queue is handled quickly while an idle queue produces very little API traffic. Scanner findings can be imported in bulk into the Claude queue for review or execution.

5.4 Current Operator Console

The current deployed UI at `/app/claude` confirms the authenticated SystemAdmin session and gives access to the operator surface. It is a lightweight restored shell and should be expanded into the richer Claude task board described by the API and README.

6. Dashboards, Audit, Notifications, and Files

6.1 Dashboard Metrics

The dashboard API computes total tasks, queued tasks, running tasks, completed-today count, failed-today count, dead-letter count, active workers, idle workers, offline workers, average duration, and success rate. Queue health groups work by queue name, and trend endpoints expose failure and completion trends.

6.2 Audit and Self-Healing

The audit domain stores audit runs, findings, recommendations, cleanup tasks, and optimization suggestions. The audit worker described by the README scans repeat failures, dead-letter backlog, stuck running tasks, orphaned artifacts, and queue backlog. High or critical findings can create self-healing tasks, closing the loop from detection to remediation.

6.3 Notifications

The notifications API lists notifications, marks individual notifications as read, marks all as read, dismisses notifications, sends notifications, and manages notification preferences. A SignalR notifications hub supports user-targeted real-time delivery.

6.4 File Handling

The files API supports upload, listing, and download. The domain contains file items, folders, versions, and access logs, allowing the application to evolve toward auditable file storage for task inputs, outputs, and attachments.

6.5 Webhooks

Webhook endpoints can be listed, created, updated, deleted, and secret-rotated. A background delivery worker polls pending events every 10 seconds, posts JSON payloads to active endpoints, signs each request with HMAC-SHA256, retries failures with exponential backoff, and marks delivery success or failure.

7. Administration and Tenant Management

7.1 Admin API

System administration endpoints cover users, feature flags, plans, companies, and roles. Admins can list users, get a user, create users, update users, reset passwords, delete users, manage feature flags, manage plans, manage companies, and list roles.

7.2 Companies and Tenancy

Companies have names, slugs, legal names, contact fields, country, timezone, active status, plan relationships, branding, settings, and users. Task and dashboard queries can scope results by the current user's company where applicable.

7.3 Plans and Billing Domain

The domain model includes plans, company plans, subscriptions, invoices, payments, credit ledgers, and usage records. This gives MADai a foundation for commercial subscription tiers, credit accounting, usage limits, and invoices, even where the current UI does not yet expose the full billing experience.

7.4 System Settings and Feature Flags

Settings provide a key/value store used by operator controls such as worker active flags and deployment timing. Feature flags are managed through the admin API and can support staged rollouts, beta features, tenant-specific enablement, or kill-switches.

8. API Reference by Capability

Capability	Representative endpoints
Auth	POST /api/v1/auth/login, refresh, logout, register, forgot-password, reset-password, MFA start/verify/disable, change-password, profile
Tasks	GET/POST/PUT /api/v1/tasks, cancel, retry, import-bulk
Task collaboration	Comments under /api/v1/tasks/{taskId}/comments; recommendations under /api/v1/task-recommendations
Workers	POST /api/v1/worker/register, heartbeat, claim, progress, log, complete, fail; admin list/drain/deactivate
Dashboard	Overview, queues, workers, failure trends, completion trends
Claude tasks	List, next, import-bulk, get, create, update, delete, attachment upload/delete
Prompt templates	List, create, update, delete Claude prompt templates
Audit	Findings, runs, recommendations
Files	Upload, list, download
Notifications	List, read, read-all, dismiss, send, preferences
Admin	Users, feature flags, plans, companies, roles
Webhooks	List, create, update, delete, rotate secret
Persistent workers	Repository intelligence, task injection, session rotation, native processes
System	Ping, health, readiness, Swagger in development

8.1 SignalR Hubs

Hub	Purpose
/hubs/tasks	Task lifecycle and Claude task update events.
/hubs/workers	Worker heartbeat and status changes.
/hubs/notifications	User-targeted notifications.
/hubs/dashboard	Aggregate dashboard refresh signals.

9. Operational Runbook

9.1 DEV Startup

Local development uses the API on <http://localhost:3011> and the frontend on <http://localhost:4211>. The API can use LocalDB for development. The frontend is an Angular standalone app. pnpm is configured to use the shared store at <C:/Code/.pnpm>.

9.2 Production Deployment

The deploy scripts publish the .NET API, inject environment variables into web.config, and upload API and frontend files by FTP to the Plesk/1-grid host. The current production hosts are <https://madaiaiapi.madprospects.com> and

<https://madai.madprospects.com>.

9.3 Health Checks

- GET /health confirms application health.
- GET /health/ready checks readiness dependencies such as SQL.
- The frontend root and /login should return the Angular application.
- Unauthenticated Hangfire dashboard access should be denied.

9.4 Common Operator Checks

- 1 Confirm the API health endpoint returns 200.
- 2 Sign in through the frontend and verify the expected role appears.
- 3 Create or inspect tasks and verify queue counts.
- 4 Confirm workers heartbeat and capacity look sane.
- 5 Inspect failures and dead-letter counts before triggering retries.
- 6 Check webhook delivery status when downstream integrations are not receiving events.
- 7 Review audit findings and decide whether to apply, dismiss, or convert them to work.

10. Limitations and Next UI Work

The backend is materially broader than the current restored frontend. The frontend should be expanded to expose dashboards, task management, workers, audit findings, notifications, files, admin, settings, webhooks, prompt templates, and a full Claude task board. Until those screens are rebuilt, many functions are API-first and require Swagger, direct API clients, scripts, or future UI work.

10.1 Known Technical Caveats

- The baseline task executor described in the README produces deterministic output until connected to the intended Claude Code Desktop bridge.
- Hangfire is wired but recurring jobs need explicit scheduling where required.
- Production secrets must remain in environment-specific configuration, not committed files.
- The app should continue to avoid repo-local node_modules folders and use C:/Code/.pnpm for pnpm storage.